北京理工大學
BEIJING INSTITUTE OF TECHNOLOGY

第12章
Java输入输出流

BEIJING INSTITUTE OF TECHNOLOGY

本章知识点

- Java流的类层次结构
- 字节流
  - 抽象类InputStream和OutputStream
  - 文件流FileInputStream和FileOutputStream
  - 缓冲流BufferedInputStream和BufferedOutputStream
  - 数据过滤流DataInputStream和DataOutputStream
  - 打印流PrintStream
  - 序列化接口Serializable
  - 对象流ObjectInputStream和ObjectOutputStream
  - 字节数组流ByteArrayInputStream和ByteArrayOutputStream

本章知识点

- 字符流
  - 抽象类Reader和Writer
  - 转换流InputStreamReader和OutputStreamWriter
  - FileReader和FileWriter
  - BufferedReader类
  - PrintWriter类
- RandomAccessFile类

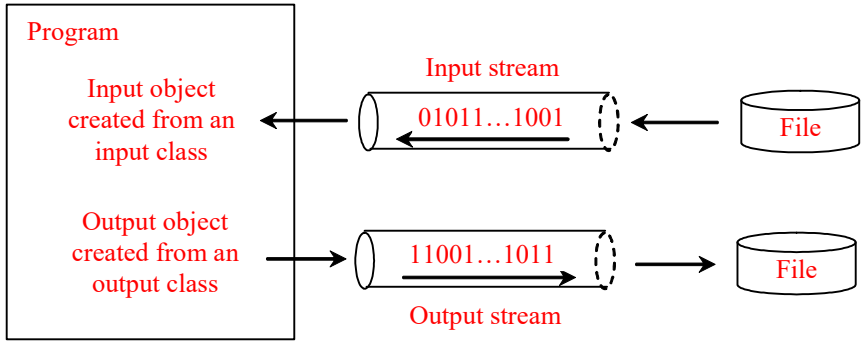
- 操作文件的类和接口

1、Java流的类层次结构

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

1.1输入输出流

- Java中的输入/输出（Input/Output，简称I/O）：程序与外部设备或其他计算机进行数据交换的过程。



- Java用“流”的方式处理I/O，对应不同类型的I/O问题，会有相应的流象提供解决方案。

按流的方向分为**输入流**和**输出流**。

按数据传输单位分为**字节流**和**字符流**。

字节流（stream）每次读写一个字节，可以读写任意文件

字符流（reader，writer）每次读写一个字符，可以方便读写文本文件

按功能分为底层的**节点流**和上层的**处理流**（或称为包装流）

- 节点流用于进行输入输出操作
- 处理流用于在节点流的基础上进行加工、转换、处理。

节点流：进行输入输出操作

类　型	字 符 流	字 节 流
File（文件）	FileReader FileWriter	FileInputStream FileOutputStream
Memory Array	CharArrayReader CharArrayWriter	ByteArrayInputStream ByteArrayOutputStream
Memory String	StringReader StringWriter	—
Pipe（管道）	PipedReader PipedWriter	PipedInputStream PipedOutputStream

http://blog.csdn.net/jingzi123456789

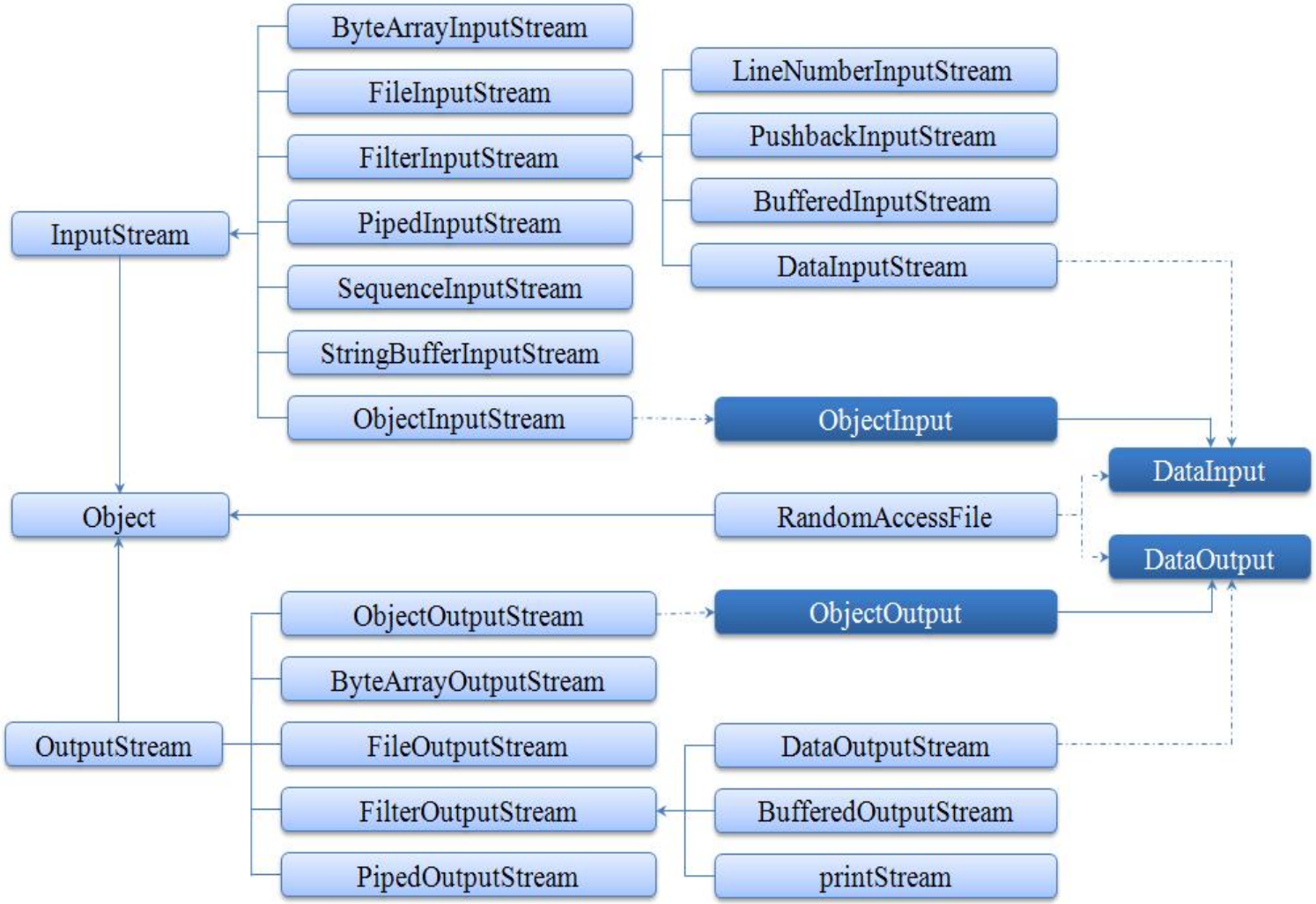
处理流：在节点流的基础上进行加工、转换、处理

处理类型	字符流	字节流
Buffering	BufferedReader BufferedWriter	BufferedInputStream BufferedOutputStream
Filtering	FilterReader FilterWriter	FilterInputStream FilterOutputStream
Converting between bytes and character	InputStreamReader OutputStreamWriter	
Object Serialization	—	ObjectInputStream ObjectOutputStream
Data conversion	—	DataInputStream DataOutputStream
Counting	LineNumberReader	LineNumberInputStream
Peeking ahead	PusbackReader	PushbackInputStream
Printing	PrintWriter	PrintStream

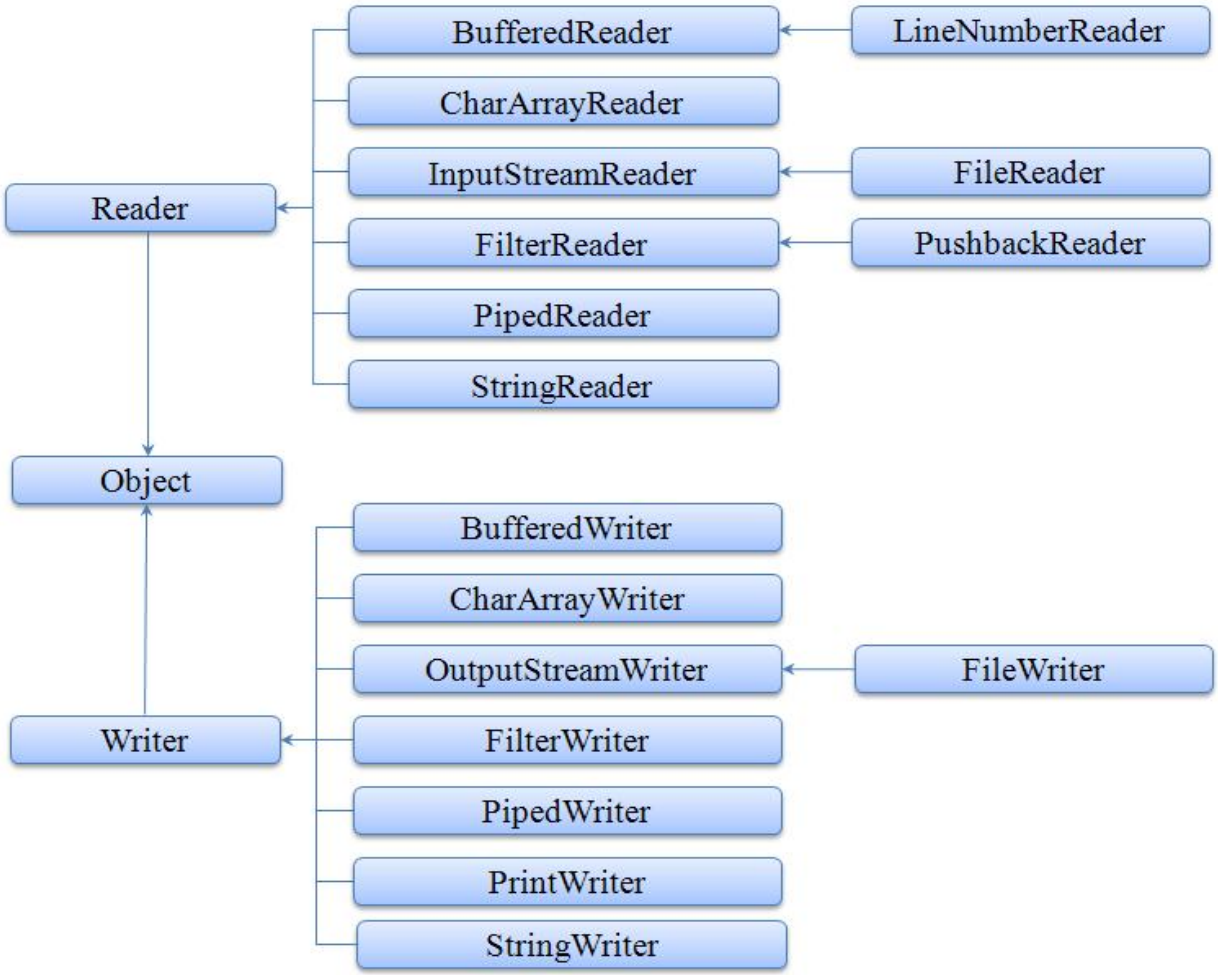
http://blog.csdn.net/qq123456789

（1）Buffering缓冲流：在读入或写出时，对数据进行缓存
（2）Filtering 滤流：在数据进行读或写时进行过滤。
（3）Converting between Bytes and Characters 转换流：按照一定的编码/解码标准将字节流转换为字符流，或进行反向转换 InputStreamReader、OutputStreamWriter。 -
（4）Object Serialization 对象流 ： ObjectInputStream、ObjectOutputStream
（5）DataConversion数据流： 按基本数据类型读、写 DataInputStream、DataOutputStream 。 -
（6）Printing打印流： 包含方便的打印方法 ： PrintWriter、PrintStream

1.2字节流层次结构


ByteArrayInputStream
FileInputStream
FilterInputStream
PipedInputStream
SequenceInputStream
StringBufferInputStream
ObjectInputStream
LineNumberInputStream
PushbackInputStream
BufferedInputStream
DataInputStream
InputStream
ObjectInput
DataInput
Object
RandomAccessFile
DataOutput
ObjectOutputStream
ObjectOutput
ByteArrayOutputStream
OutputStream
FileOutputStream
DataOutputStream
FilterOutputStream
BufferedOutputStream
PipedOutputStream
printStream

1.2字符流层次结构

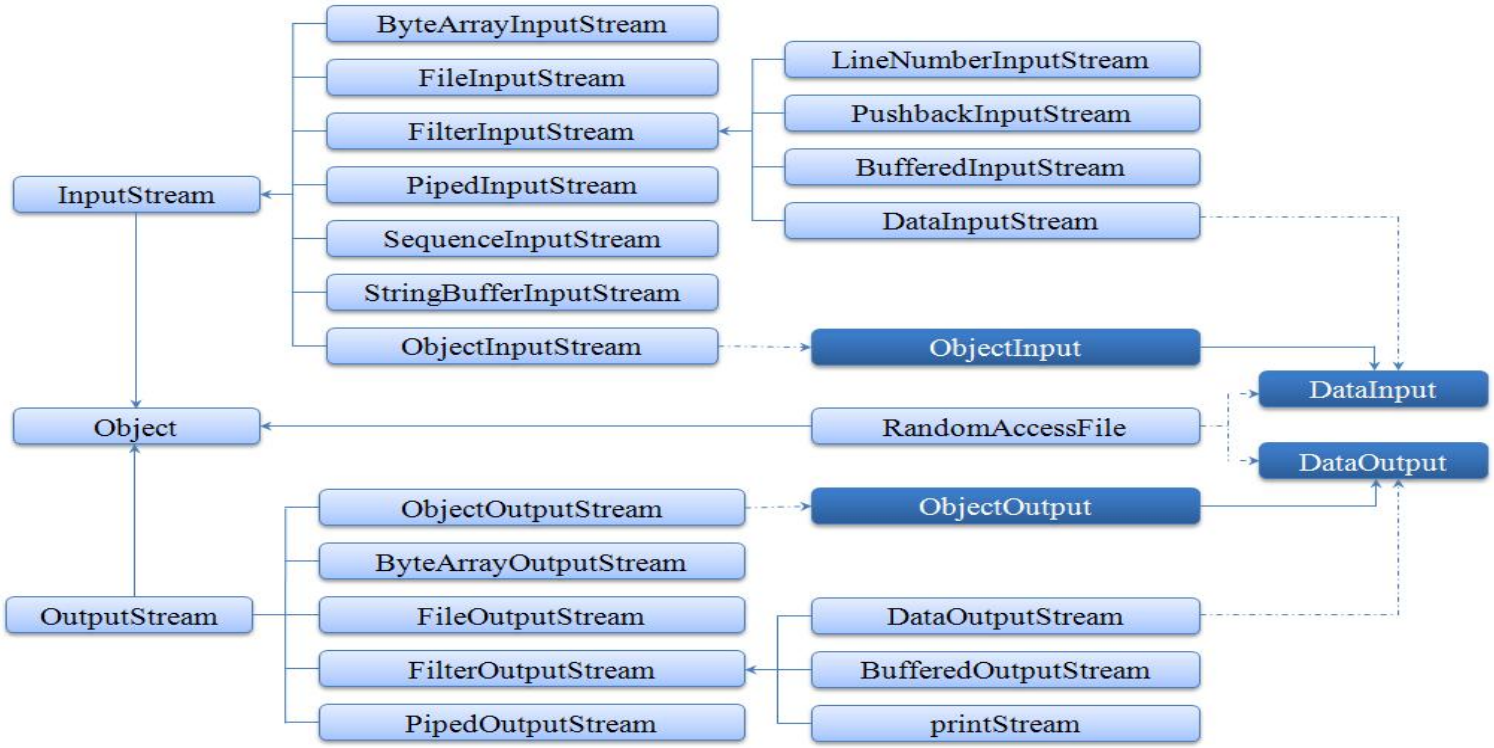

BufferedReader
LineNumberReader
CharArrayReader
InputStreamReader
FileReader
Reader
FilterReader
PushbackReader
PipedReader
StringReader
Object
BufferedWriter
CharArrayWriter
OutputStreamWriter
FileWriter
Writer
FilterWriter
PipedWriter
PrintWriter
StringWriter

2 字节流

- 字节流以8位的字节为读写单位。
- 字节流体系：以抽象类InputStream和OutputStream作为顶层向下包含了众多子类成员，可以按照输入流/输出流的方式完成各种基本数据类型数据、数组、字符串、对象、文件等的读入和写出。

1、InputStream类

- 用来表示那些从不同数据源产生输入的类。



2.1 抽象类InputStream和OutputStream

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- InputStream中的方法
  - int read()
  - int read(byte[] b)
  - int read(byte[] b, int off, int len)
  - int available()：返回流中可用字节数。
  - void close()：关闭输入流，并释放与该流关联的所有系统资源。

2.1 抽象类InputStream和OutputStream

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- OutputStream类中的方法
  - write(int b)
  - write(byte[] b)
  - write(byte[] b, int off, int len)
  - void close()：关闭输出流。
  - void flush()：强制刷新输出缓冲区内容，将输出缓冲区内容写入流。

- FileInputStream和FileOutputStream：基于字节、广泛用于操作文件。
  - FileInputStream类用于读取一个文件，FileOutputStream类用于将数据写入一个文件。

FileStream.java

- 它们都会打开一个到实际文件的连接，并创建一个文件输入流，如果该文件不存在，或者它是一个目录，或者因为其他原因而无法打开，会抛出FileNotFoundException异常。前者文件通过路径名name指定，后者通过File对象指定。

12.2.2 文件流FileInputStream和FileOutputStream

编写一个对指定文件进行加密复制的程序。

- 分析：复制文件就是将一个文件读出再写入另一个文件的过程，无论文件是何类型，都可以按照字节一一读出，一一写入，所以使用FileInputStream和FileOutputStream可以实现任意文件的复制。
- read()方法每次从文件中读取一个字节，范围从0到255间，如果已到达文件的末尾，则返回-1，所以循环读取的条件为：检测读到的数据是否为-1。
- 所谓加密复制就是在写入数据前对其进行加密处理，可以采取异或运算。

12.2.2 文件流FileInputStream和FileOutputStream

【例12-2】利用try-with-resource处理机制重写文件加密复制程序。

- 分析：传统的输入输出流的关闭过程代码繁琐，从Java SE 7引入了try-with-resource处理机制。
- 如果一个类实现了AutoClosable接口，那么创建这个类的对象就可以写在try-catch的try后面的括号中，并且能在try代码块正常结束前或者因发生异常要抛出异常前自动执行close()方法，保证资源正常关闭，不再需要书写finally代码块。

12.2.2 文件流FileInputStream和FileOutputStream

【例12-2】利用try-with-resource处理机制重写文件加密复制程序。

- Java SE 7后java.io中的类都实现了AutoClosable接口，可以直接使用try-with-resource。

PREV CLASS NEXT CLASS FRAMES NO FRAMES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3

java.io

Class FileInputStream

```
java.lang.Object
    java.io.InputStream
        java.io.FileInputStream
```

All Implemented Interfaces:

Closeable, AutoCloseable

12.2.2 文件流FileInputStream和FileOutputStream

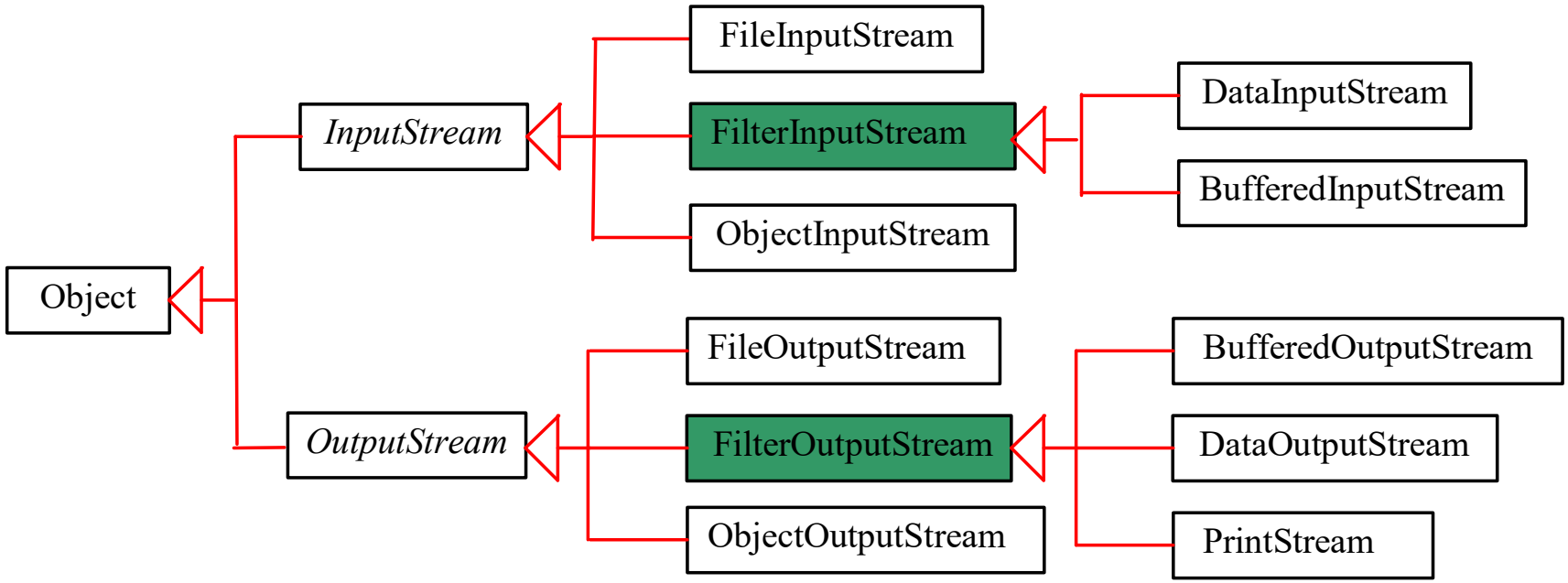
在try关键字后面的括号中声明并初始化一个或多个资源，这些资源必须实现了AutoCloseable接口

```
try (ResourceType resource = new
ResourceType(args...)) {
    // 代码块
} catch (ExceptionType e) {
    // 异常处理
}
```

在代码块执行完毕之后，Java会自动调用这些资源的close()方法，释放资源

12.2.2.3FilterInputStream/FilterOutputStream

Filter streams are streams that filter bytes for some purpose.



12.2.4 缓冲流BufferedInputStream和BufferedOutputStream

- BufferedInputStream和BufferedOutputStream是FilterInputStream和FilterOutputStream的子类，为普通的字节流增加了缓冲区功能，将InputStream和OutputStream对象包装为一个带缓冲的字节流。

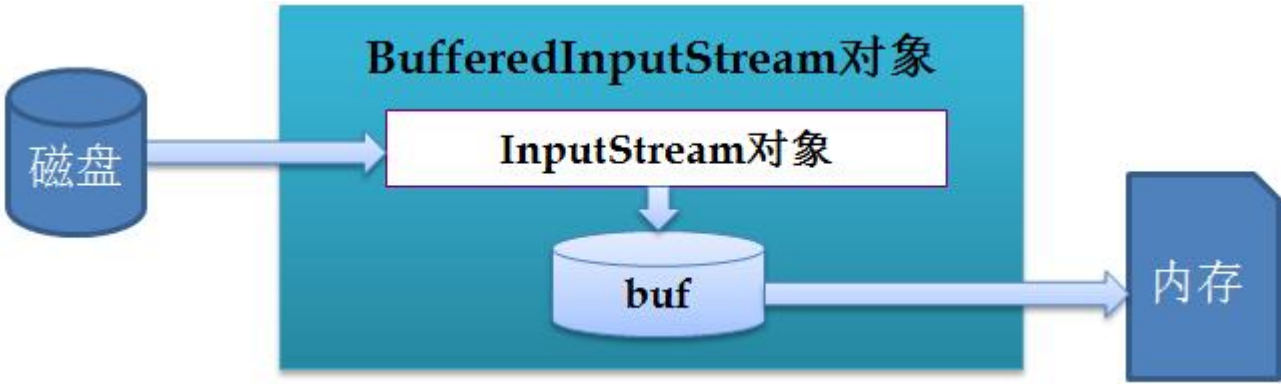
- 原理：因为内存的读写速度快，而磁盘的读写速度慢，二者间的数据传输堵塞严重。所以，为了减少对磁盘的存取，通常在内存和磁盘间建立一个缓冲区，从磁盘中读数据时一次读入一个缓冲区大小的数量，数据写入磁盘时也是先将缓冲区装满后，再将缓冲区数据一次性写入到磁盘，由此提高了文件存取的效率。

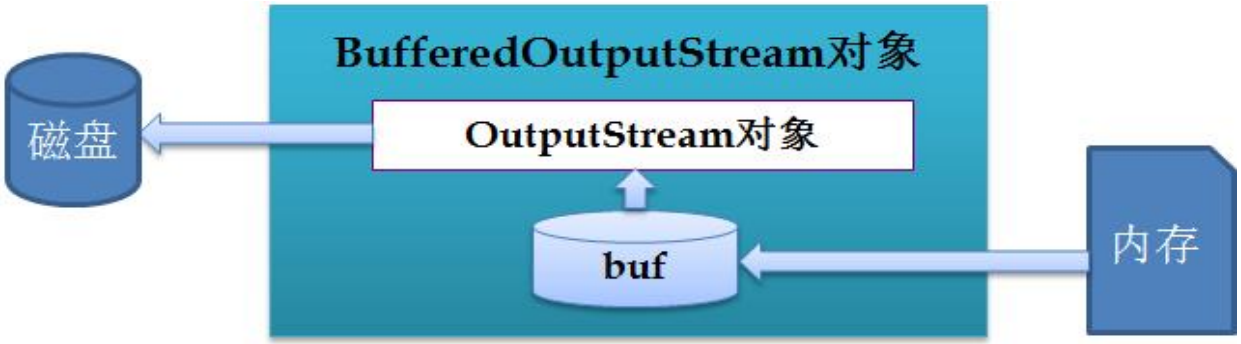
12.2.4 缓冲流BufferedInputStream和BufferedOutputStream

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- BufferedInputStream的构造方法：
  - BufferedInputStream(InputStream in)：包装InputStream对象、创建BufferedInputStream对象，并创建一个默认大小（8192字节）的字节数组做缓冲区。
  - BufferedInputStream(InputStream in, int size)：包装InputStream对象、创建指定缓冲区大小的BufferedInputStream对象。

12.2.4 缓冲流BufferedInputStream和BufferedOutputStream





12.2.4 缓冲流BufferedInputStream和BufferedOutputStream

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 【例12-3】对比带缓冲区的文件复制与不带缓冲区文件复制的性能。
  - 通过完成相同的复制工作所花费的时间对比得到带缓冲区与不带缓冲区间的差别。

12.2.4 缓冲流BufferedInputStream和BufferedOutputStream

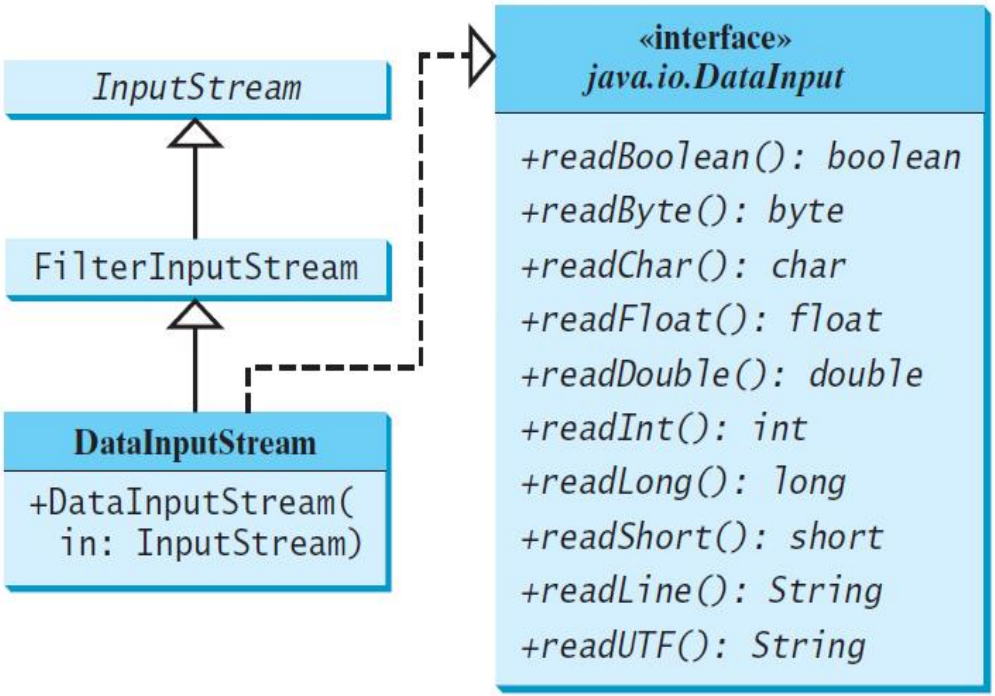
- 【 关于flush()刷新】
- 对于BufferedOutputStream，只有缓冲区满时，才会将数据真正送到输出流。
- 有些情况下，就需要人为地调用flush()方法将尚未填满的缓冲区中的数据送出。例如，数据已经读取完毕，但缓冲区尚未装满，这时必须由程序调用flush()方法强制刷新缓冲区。

12.2.5 数据过滤流DataInputStream和DataOutputStream

- 数据过滤流类DataInputStream和DataOutputStream是FilterInputStream和FilterOutputStream的子类。
- 它们将字节数据按照指定形式结合成有意义的基本数据类型数据、以及String类型数据。

12.2.5 数据过滤流DataInputStream和DataOutputStream

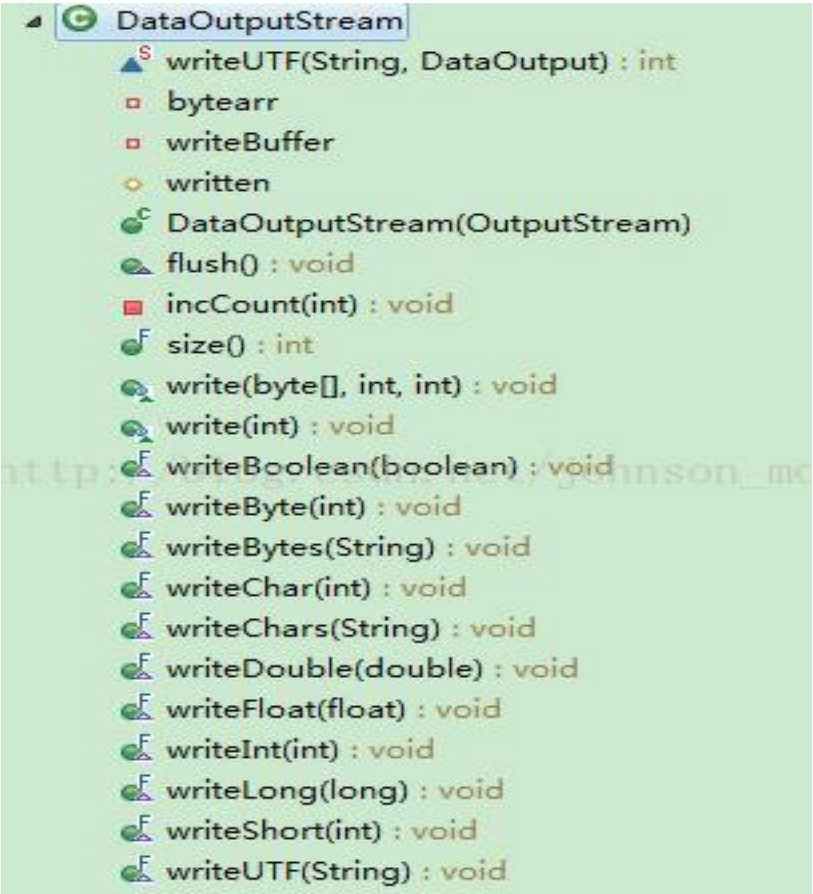
- DataInputStream除父类InputStream中的3个read()方法外，还有8种基本数据类型数据的读方法，如果读取过程中输入流已到达文件末尾，则抛出EOFException异常。



Reads a Boolean from the input stream.	
Reads a byte from the input stream.	
Reads a character from the input stream.	
Reads a float from the input stream.	
Reads a double from the input stream.	
Reads an int from the input stream.	
Reads a long from the input stream.	
Reads a short from the input stream.	
Reads a line of characters from input.	
Reads a string in UTF format.	

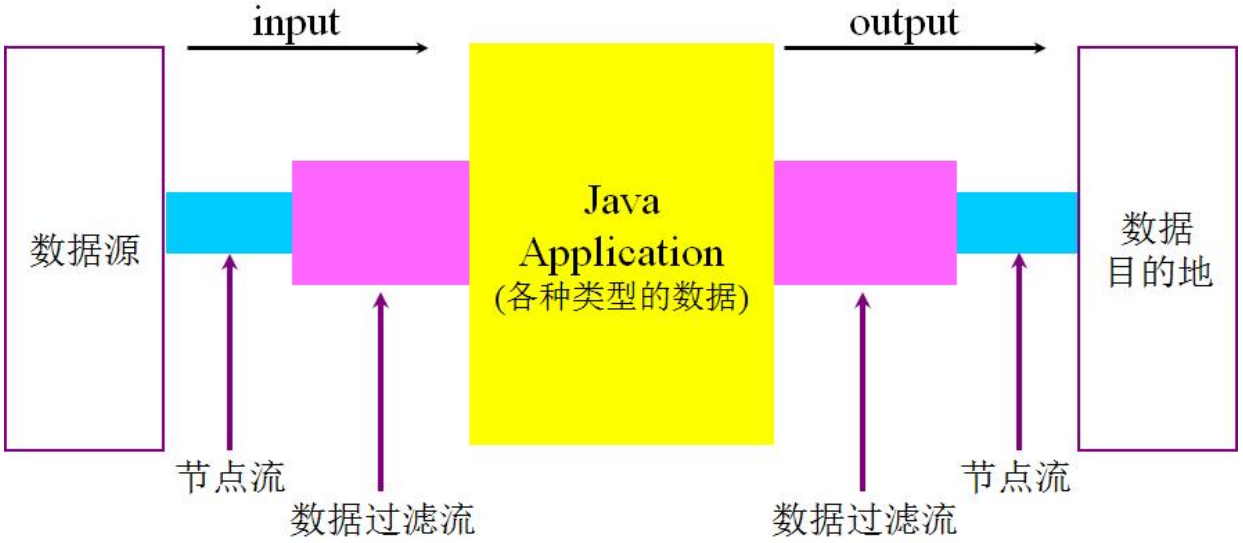
12.2.5 数据过滤流DataInputStream和DataOutputStream

- DataOutputStream除父类OutputStream中的和3个write()方法外，还有8种基本数据类型数据的写方法：



12.2.5 数据过滤流DataInputStream和DataOutputStream

- 它们的构造方法就是对已有的InputStream和OutputStream对象进行包装：DataInputStream(InputStream in)，DataOutputStream(OutputStream out)



12.2.5 数据过滤流DataInputStream和DataOutputStream

【例12-4】使用数据过滤流将1000-2000间的素数写到文件中持久化保存；将这些素数从文件读出、按每行10个的形式打印。

- 分析：使用自定义方法boolean isPrime(int)判断某数是否是素数。找到素数后，将其用writeInt()方法写入文件。读取文件时，每次使用readInt()方法读取一个int。

12.2.6 对象流与序列化接口Serializable

- 有时希望将以对象方式存在于内存中的数据存储至文件（持久化到文件），需要时再将其从文件中读出还原为对象，或者在网络上传送对象，
- 使用Java提供的对象流ObjectInputStream和objectOutputStream。

12.2.6 对象流与序列化接口Serializable

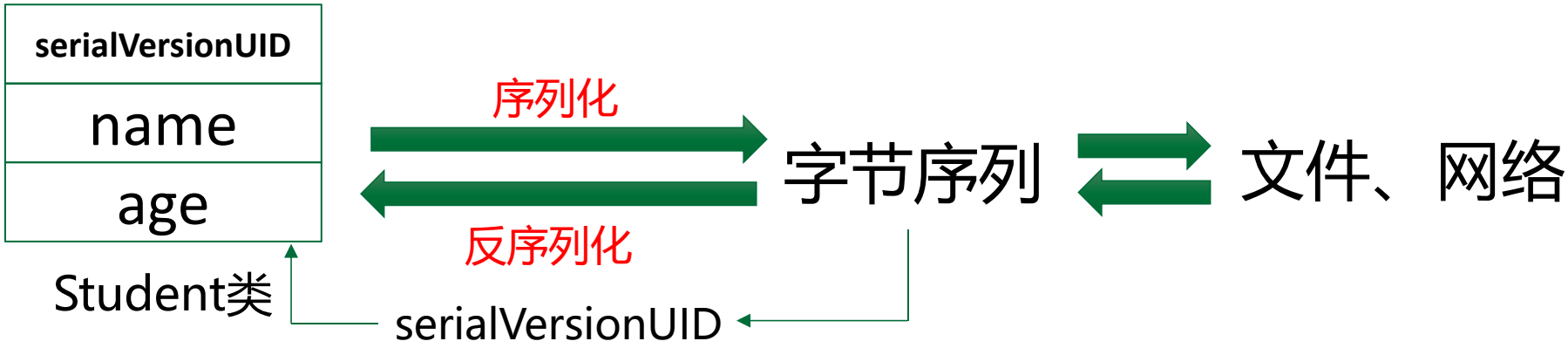
北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 如果对象需要被持久化到文件，或者在网络上传送对象，则定义该对象的类必须实现**Serializable**接口。
  - 把Java对象转换为字节序列的过程称为对象的序列化，把字节序列恢复为Java对象的过程称为对象的反序列化。

```
public class Foo
implements
java.io.Serializable {
  private int v1;
  private static double v2;
  private transient A v3 =
new A();
}
```

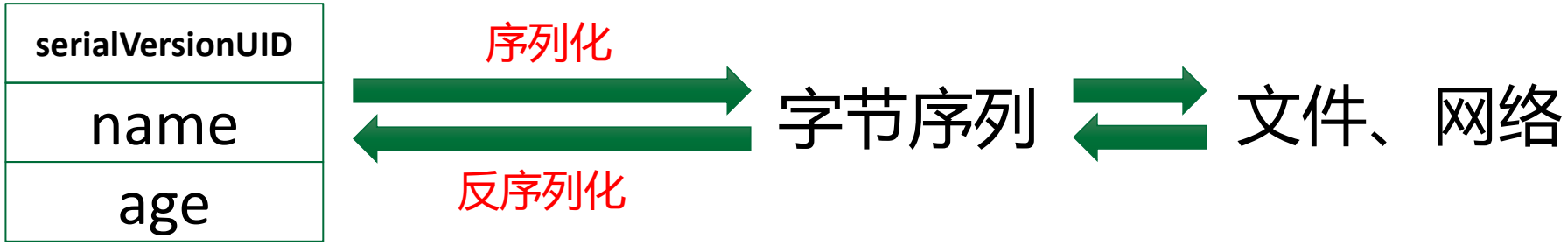
12.2.6 对象流与序列化接口Serializable

- JVM使用serialVersionUID来验证版本一致性。在进行反序列化时，JVM会把传来的字节流中的serialVersionUID与本地相应类的serialVersionUID进行比较，如果相同就认为是一致的，可以进行反序列化，否则就会出现序列化版本不一致的InvalidCastException异常。



12.2.6 对象流与序列化接口Serializable

- 支持序列化的类使用常量serialVersionUID标识可序列化对象的版本，从而维持版本信息的一致。
- 为了提高serialVersionUID的独立性和确定性，强烈建议显式定义serialVersionUID，为它赋予明确的取值。



Student类

```
public class Student implements Serializable{
    private static final long serialVersionUID = -
    7108027765951316257L;
}
```

12.2.6 对象流与序列化接口Serializable

对象可序列化：必须满足如下要求：
1、该类实现了Serializable接口。
2、该类的所有参与序列化的成员变量也必须是可序列化的（即定义成员变量的类型也要实现Serializable，如String、数据类型等），即使类实现了Serializable接口，如果有定义没有实现Serializable类型的成员变量，则这个类的对象也不能序列化。

当一个可序列化类有多个父类时（包括直接和间接父类），这些父类要么有无参的构造器，要么也是可序列化的——否则反序列化时抛出InvalidClassExcption异常。如果父类是不可序列化的，只带有无参数的构造器，则该父类中定义的成员变量值不会序列化到二进制流中

12.2.6 对象流与序列化接口 Serializable

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

【例12-5】将一个Student对象持久化到文件，并从文件读出、打印。

注意：

1、当程序多次writeObject同一个对象时，实际上并未输出多个对象，程序多次调用readObject时恢复的全部是同一个对象。

2、static修饰的是是类变量，不会被序列化。

3、用transient修饰的数据成员不会被序列化

Example:12-6 中的Student

对象引用的序列化原理：当程序第一次序列化某个对象时（包括用writeObject输出、或关联序列化时），系统才会真正将该对象的数据输出到磁盘或者网络，并为该序列化对象分配一个版本化编号，以后如果再次序列化该对象，并不会真正输出该对象数据，而是输出版本化编号。

12.2.7 字节数组流ByteArrayInputStream和ByteArrayOutputStream

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 流的来源或目的地不一定是文件，也可以是内存中的一段空间。
  - 内存虚拟文件就是把内存中的数据缓存区虚拟成一个文件，原来应该写入到磁盘文件的内容，可以写入内存中；原来应该从磁盘文件中读取的内容，也可以从内存中读取。这样可以大大提高应用程序的性能和效率。

- 在Java中定义了ByteArrayInputStream和ByteArrayOutputStream，用于以I/O流的方式来完成对字节数组内容的读写，来支持类似内存虚拟文件的功能。
- ByteArrayInputStream和ByteArrayOutputStream作为与内存这个物理存储打交道的节点流，它们可以被处理流所包装，从而用更丰富的操作使用字节数组流中的字节数据。

12.2.7 字节数组流ByteArrayInputStream和ByteArrayOutputStream

- 构造方法：
  - ByteArrayInputStream(byte[] buf)：使用buf作为其缓冲区数组创建一个ByteArrayInputStream对象。
  - ByteArrayInputStream(byte[] buf, int offset, int length)：使用buf数组从offset位置开始的length长度空间作为缓冲区创建ByteArrayInputStream。
  - ByteArrayOutputStream()：创建一个新的字节数组输出流。
  - ByteArrayOutputStream(int size)：创建一个新的字节数组输出流，它具有指定大小的缓冲区容量。

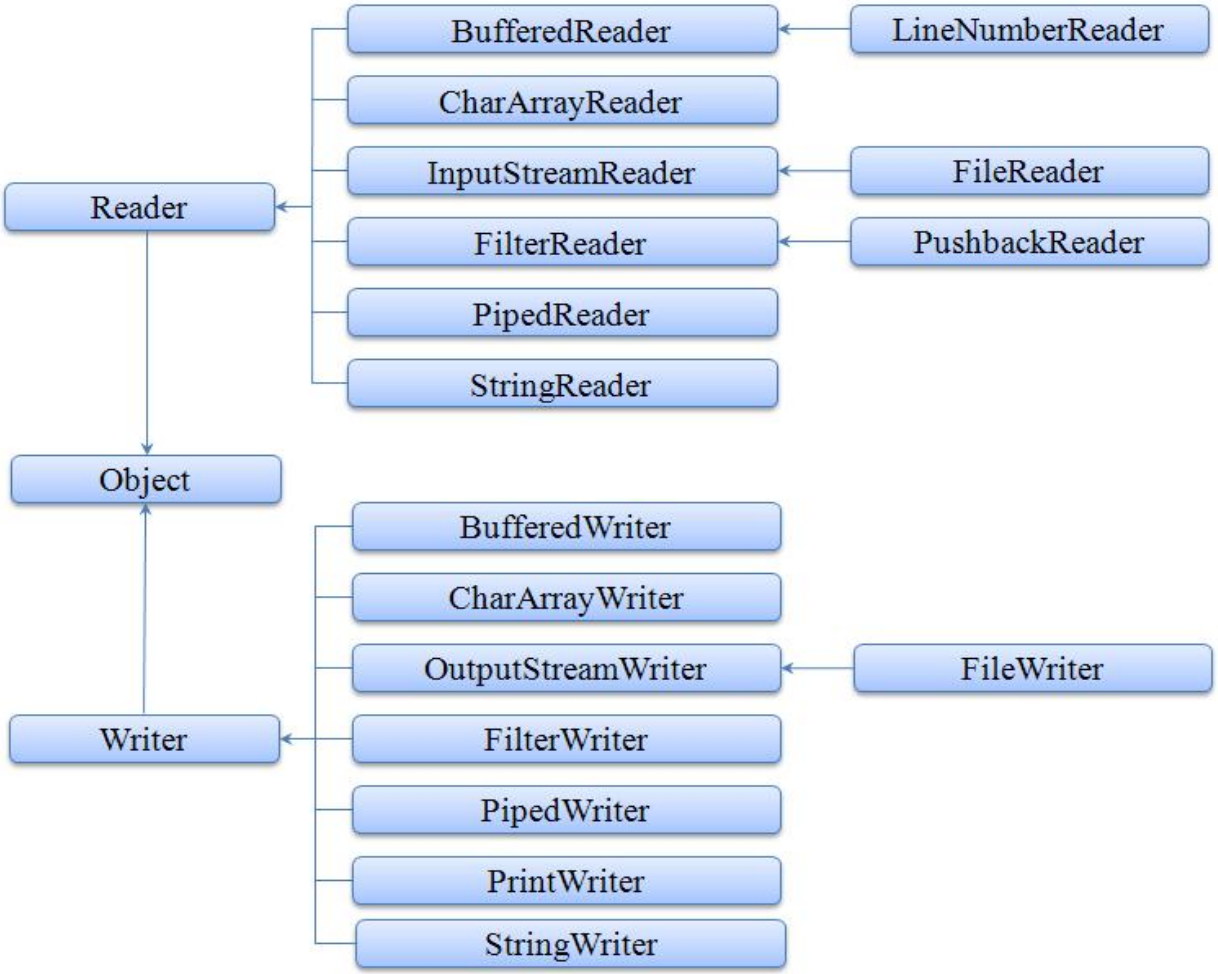
【例12-6】实现对象的深复制。

- 深复制的前提：对象以及对象内部所有引用到的对象都是可序列化的，即都实现了Serializable接口。
- 将对象持久化到文件中再读出可以获得原对象的一个拷贝，所以利用这个方法可以深复制对象。

12.3 字符流

- 为了便于读写字符型数据，让Java的I/O流都支持Unicode编码，并允许使用其他字符编码方案，Java 1.1的类库中增加了Reader和Writer继承层次结构，分别表示字符输入流和字符输出流。

12.3.1 抽象类Reader和Writer



12.3.1 抽象类Reader和Writer

- Reader类中3个基本的读取数据的方法：
  - int read()：从输入流中读取单个字符。因为Java采用Unicode编码，每个字符分配2个字节的存储空间，所以read()方法将读取2个字节，返回所读取的字符数据的Unicode编码。
  - int read(char[] cbuf)：从输入流中最多读取cbuf.length个字符的数据，并将其存储在字符数组cbuf中，返回实际读取的字符数。
  - int read(char[] cbuf, int off, int len)：从输入流中最多读取len个字符的数据，并将其存储在字符数组cbuf中，从数组的off位置开始存储。
  - 所有read()方法在读取流数据时，如果已到达流的末尾，则返回-1。

12.3.1 抽象类Reader和Writer

- Writer类中定义的向输出流写出数据的方法既包括写出字符、字符数组的，同时也包括写出字符串的，如下：
  - void write(int c)：将指定的字符输出到输出流。
  - void write(char[] cbuf)：将字符数组cbuf中的数据输出到指定输出流
  - 。
  - void write(char[] cbuf, int off, int len)：将字符数组cbuf从off位置开始的len个字符输出到指定输出流。
  - void write(String str)：将str字符串中的字符输出到指定输出流。
  - void write(String str, int off, int len)：将str字符串中从off位置开始的len个字符输出到指定输出流。

12.3.2 转换流InputStreamReader和OutputStreamWriter

- 字节流和字符流的桥梁
- InputStreamReader把InputStream转换为Reader
- OuputStreamWriter将OutputStream转换为Writer。

12.3.2 转换流InputStreamReader和OutputStreamWriter

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

		I	空格	a	m	空格	中	国
ISO8859-1		49	20	61	6d	20	3f	3f
GB2312 GBK		49	20	61	6d	20	d6d0	b9fa
Unicode码点		49	20	61	6d	20	4e2d	56fd
UTF-16 大端（高前低后）	feff	0049	0020	0061	006d	0020	4e2d	56fd
UTF-8		49	20	61	6d	20	e4b8ad	e59bbd

结论：处理文本数据的时候，为防止乱码出现，需要指定字符编码或者进行编码转换。

12.3.2 转换流InputStreamReader和OutputStreamWriter

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 2．利用转换流设置字符编码

  - Reader类能将输入流中采用其他编码方式的字节流转换为Unicode字符。
  - Writer类能将内存中的Unicode字符转换为其他的编码方式的字节流，再写到输出流。
  - 在默认的情况下，Reader和Writer会在本地平台默认字符编码和Unicode编码间进行转换。。
  - 如果需要输入、输出流采用特定编码方案，可以使用InputStreamReader和OutputStreamWriter类，它们在将字节流转换为字符流的同时指定字符编码方式。

12.3.2 转换流InputStreamReader和OutputStreamWriter

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 2．利用转换流设置字符编码
  - 如果需要输入、输出流采用特定编码方案，可以使用InputStreamReader和OutputStreamWriter类，它们在将字节流转换为字符流的同时，可以指定字符编码方式。

12.3.2 转换流InputStreamReader和OutputStreamWriter

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

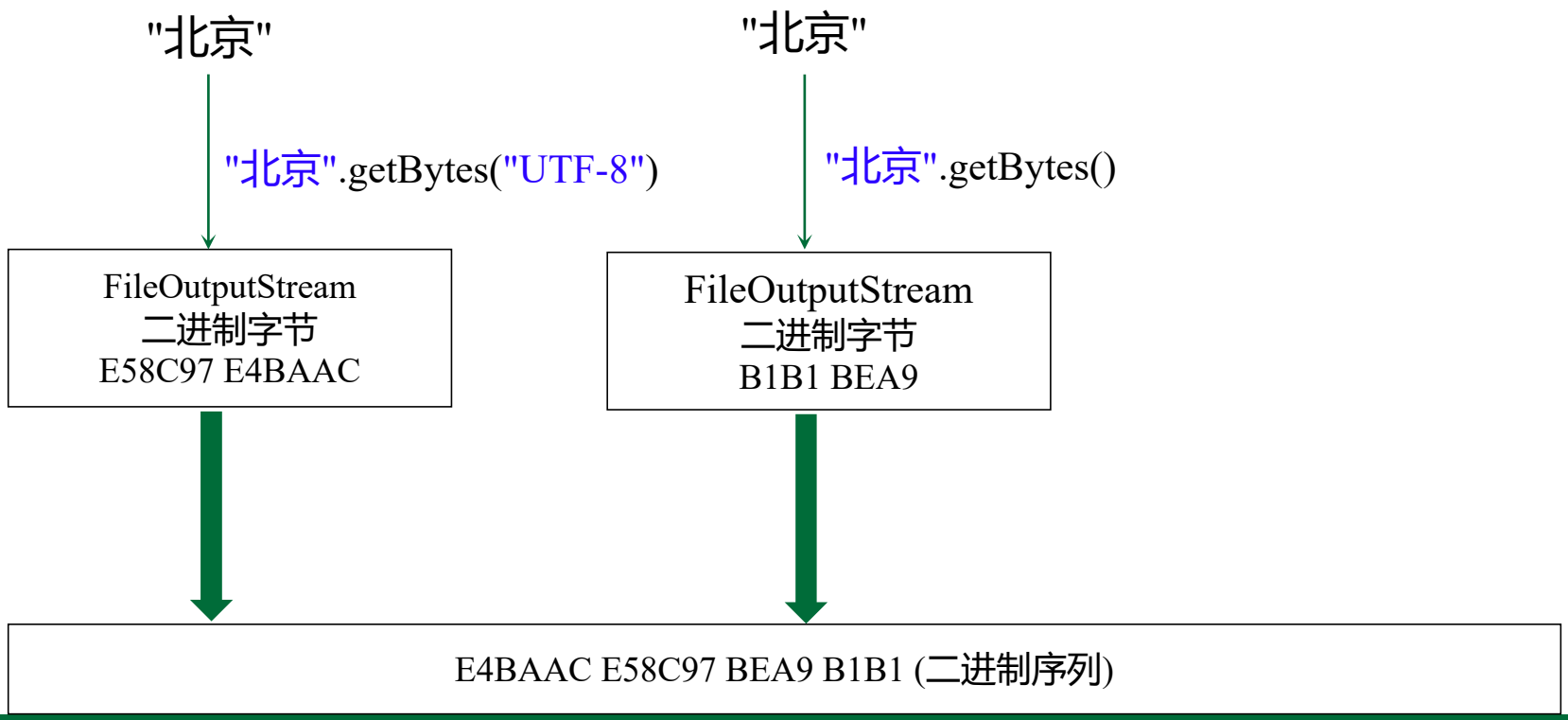
- 常用的构造方法如下：
  - InputStreamReader(InputStream in)：创建一个使用默认字符集的 InputStreamReader。
  - InputStreamReader(InputStream in,String charsetName)：创建使用指定字符集的 InputStreamReader。
  - OutputStreamWriter(OutputStream out)：创建使用默认字符编码的 OutputStreamWriter。
  - OutputStreamWriter(OutputStream out,String charsetName)：创建使用指定字符集的 utputStreamWriter。

12.3.2 转换流InputStreamReader和OutputStreamWriter

【例12-7】向文件中写入一个中文字符串，再将其从文件读出。

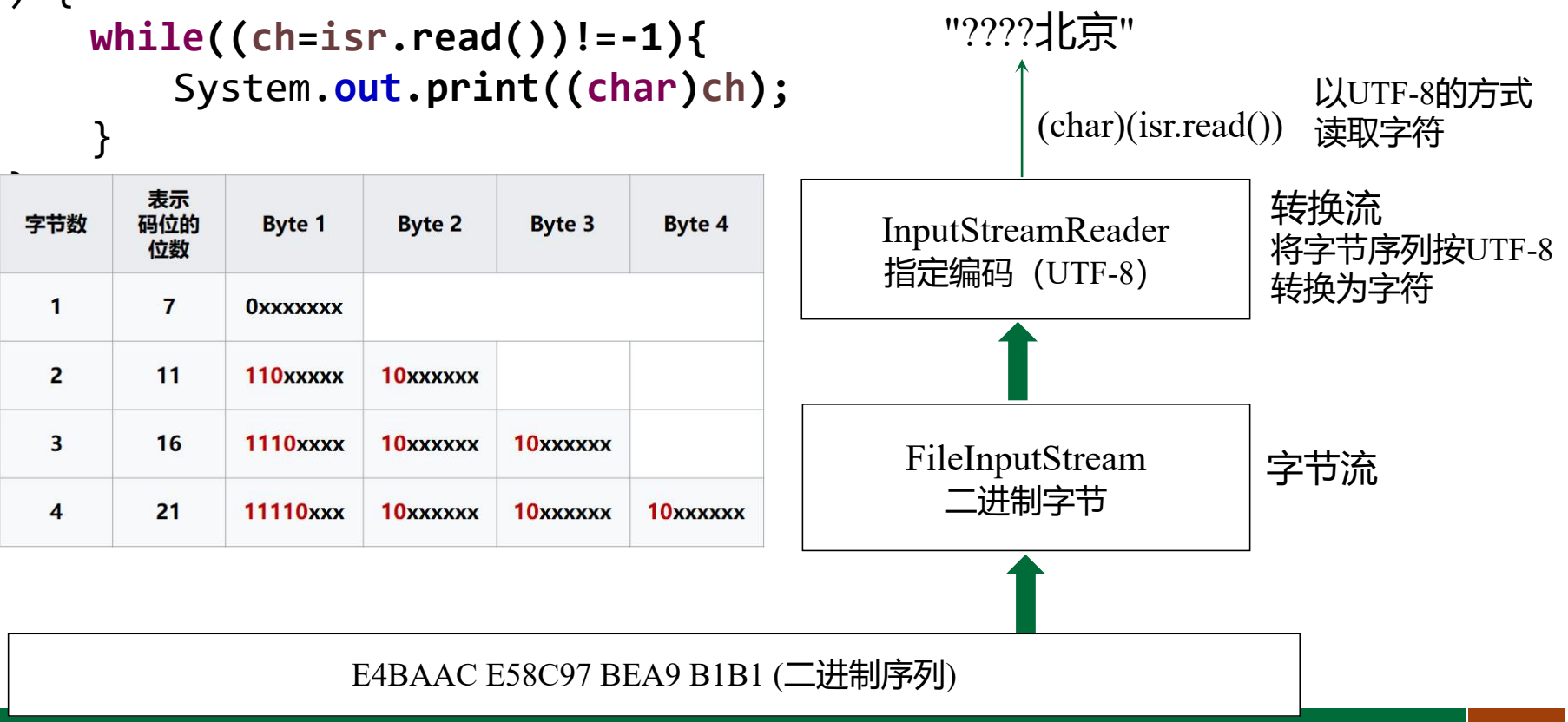
- 说明：为了演示汉字编码的使用，本例利用两种编码向输出流写入字符串“北京”，一次使用默认的本地编码方案GBK，一次指定编码方式UTF-8。从输入流读取数据时使用UTF-8编码。

```
try(
        FileOutputStream fos = new FileOutputStream(filename);
){
    fos.write("北京".getBytes());
    fos.write("北京".getBytes("UTF-8"));
}
```



```java
try(
    FileInputStream fis = new FileInputStream(filename);
    InputStreamReader isr = new InputStreamReader(fis, "UTF-8");
) {
    while((ch=isr.read())!=-1){
        System.out.print((char)ch);
    }
```

字节数	表示码位的位数	Byte 1	Byte 2	Byte 3	Byte 4
1	7	0xxxxxxx			
2	11	110xxxxx	10xxxxxx		
3	16	1110xxxx	10xxxxxx	10xxxxxx	
4	21	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx



12.3.3 FileReader和FileWriter

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 如果存取的是一个文本文件，可以直接使用FileReader和FileWriter类，它们分别继承自InputStreamReader和OutputStreamWriter。
  - FileReader类用于文本文件的读，每次读取一个字符或一个字符数组。FileWriter类用于文本文件的写，每次写入一个字符、一个数组或一个字符串。
  - 通常可以将FileReader对象看作一个以字符为单位的无格式的字符输入流，将FileWriter对象看作是以字符为单位的无格式的字符输出流。
  - FileReader和FileWriter类只能按照平台默认的字符编码进行字符的读写，若要指定编码，则还是使用InputStreamReader和OutputStreamWriter。

【例12-8】将九九乘法表保存在文本文件中。

12.3.4 BufferedReader类

- BufferedReader和BufferedWriter类都带有8192个字符的缓冲区，缓冲区的作用与BufferedInputStream和BufferedOutputStream类相同。
  - BufferedReader类还可以“文本行”为基本单位读取数据，文本行是以回车换行结束的字符序列。
  - String readLine()：从输入流中读取一行字符，如果读遇到流结束，则返回null。

Bufferwriting

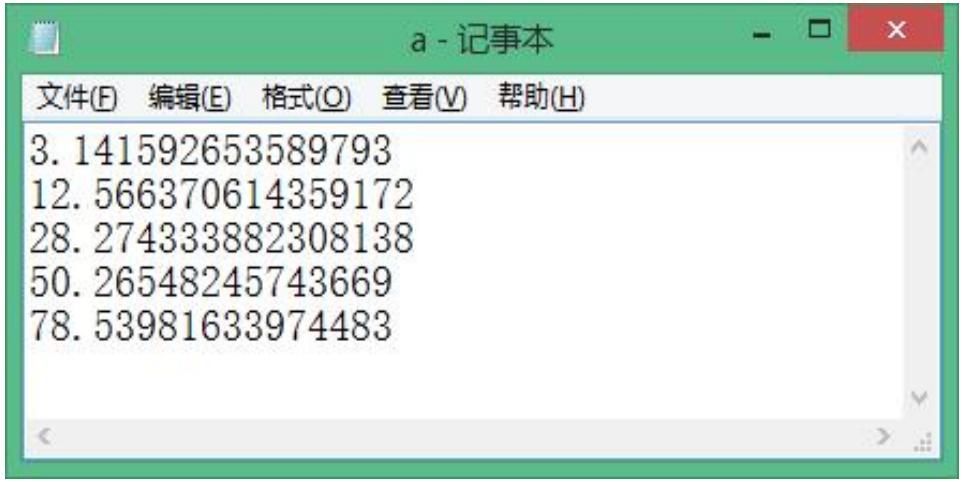
BufferReaderTest

12.3.5 PrintWriter类

PrintWriter提供了各种print方法和国际化支持

- PrintWriter提供了既能接收Writer对象又能接收OutputStream对象的构造方法，简化了输出流对象的创建过程。
- PrintWriter也可以设置缓冲区的自动刷新。PrintWriter仅在调用println()方法时自动刷新；

【例12-9】将计算得到的几个圆的面积写入一个文件；再将文件中的信息读取出来在控制台打印。



The use of layered objects（叠加对象） to dynamically and transparently add responsibilities to individual objects is referred to as the *Decorator* pattern(装饰器模式).

The Java I/O library requires many different combinations of features, which is why the decorator pattern is used.

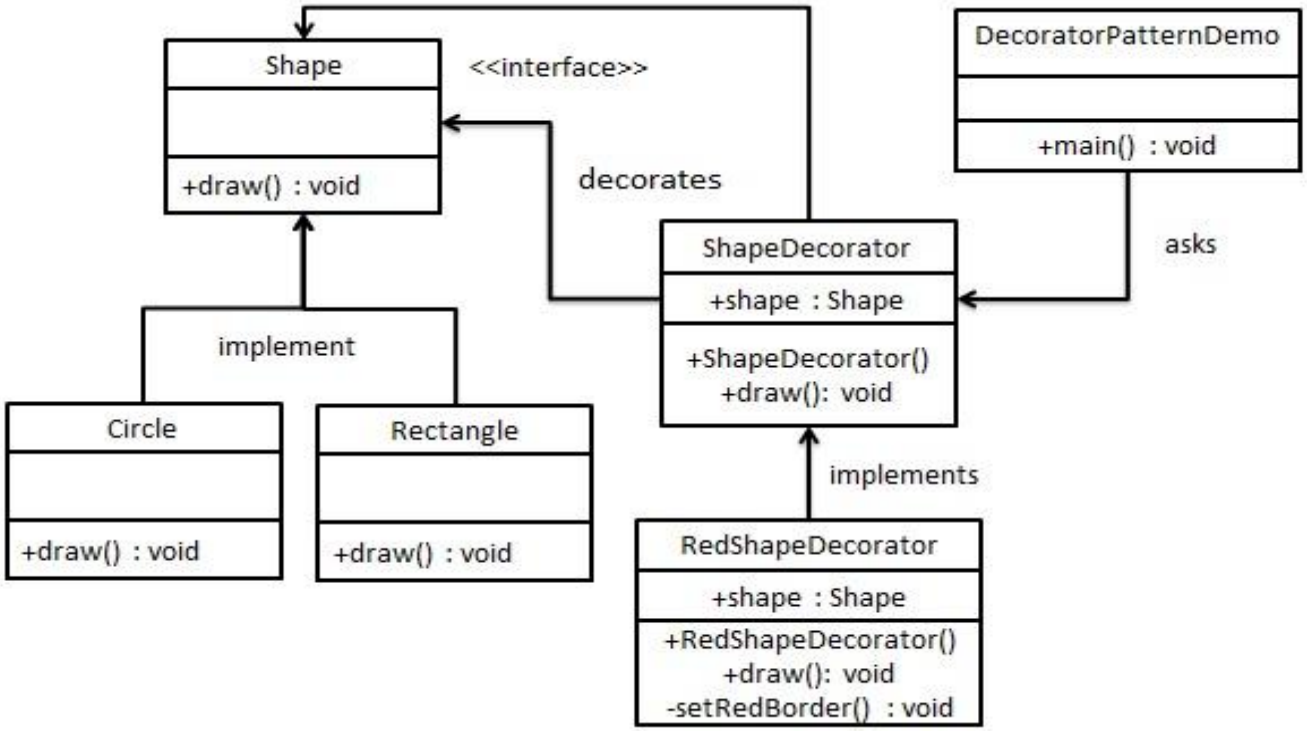
- 装饰器模式：

创建了一个装饰类，用来包装原有的类，并在保持类方法签名完整性的前提下，提供了额外的功能。

实现：

1、Component 类充当抽象角色，不应该具体实现。

2、修饰类引用和继承 Component 类，具体扩展类重写父类方法。

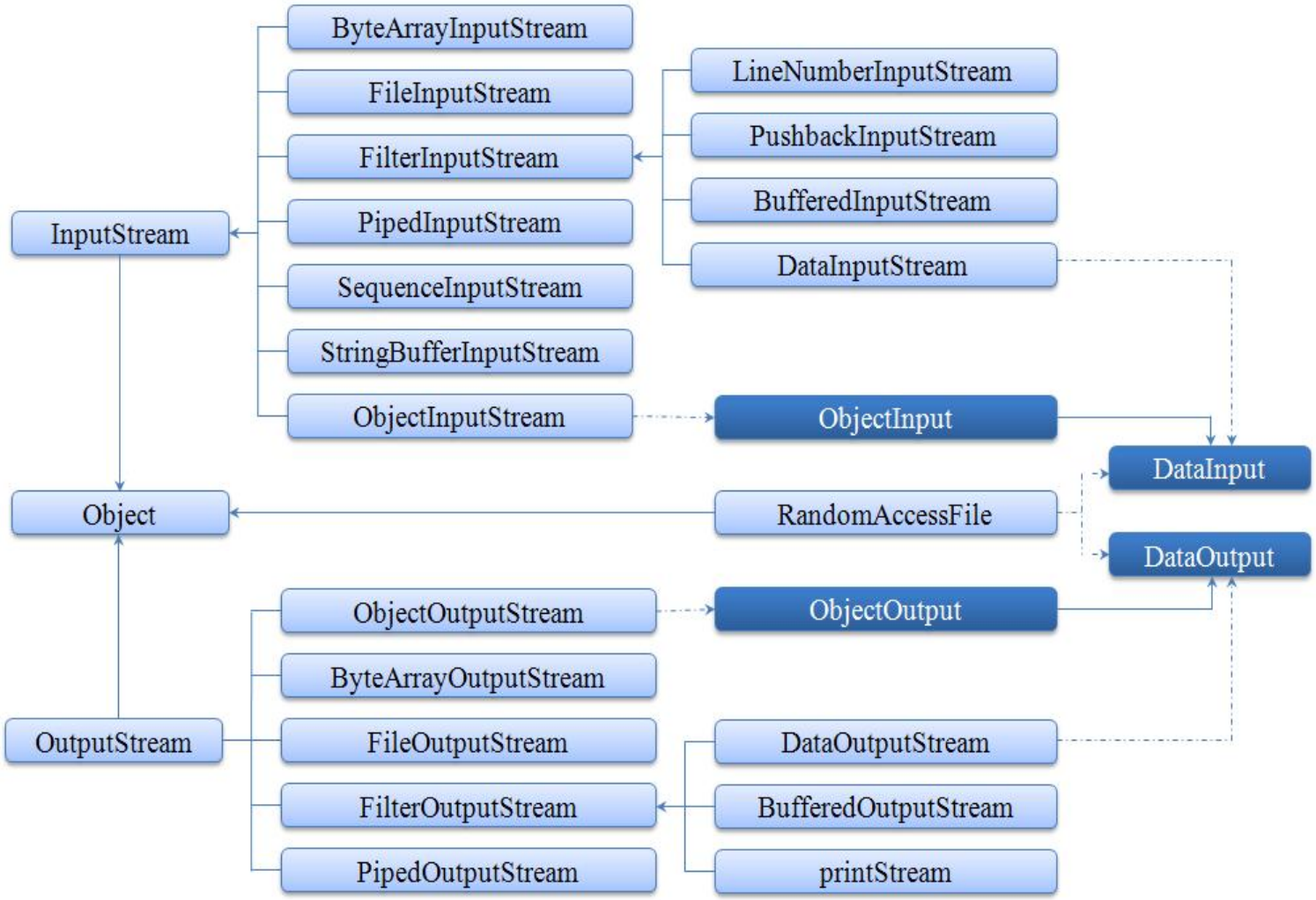

Shape
<<interface>>
+draw() : void
decorates
DecoratorPatternDemo
+main() : void
asks
ShapeDecorator
+shape : Shape
+ShapeDecorator()
+draw(): void
implement
Circle
+draw() : void
Rectangle
+draw() : void
implements
RedShapeDecorator
+shape : Shape
+RedShapeDecorator()
+draw(): void
-setRedBorder() : void

步骤

- 创建一个接口
- 创建实现接口的实体类
- 创建实现了 *Shape* 接口的抽象装饰类
- 创建扩展了 *ShapeDecorator* 类的实体装饰类
- 使用 *RedShapeDecorator* 来装饰 *Shape* 对象

Example：decoratorpattern

12.5 RandomAccessFile类



1．RandomAccessFile的特点

- RandomAccessFile类以字节为单位进行文件内容的存取。

- （1）RandomAccessFile提供了文件的“随机访问”方式，在RandomAccessFile类中定义了文件记录指针，标识当前正在读写的位置，它可以指向文件中的任意位置。
- （2）RandomAccessFile既可以读取文件的内容，也可以向文件输出数据，集读、写功能于一身。

2．RandomAccessFile中的读写方法

- 查看API文档

- 3、RandomAccessFile中的其他方法
  - long getFilePointer()：返回文件记录指针的当前位置。
  - void seek(long pos)：将文件记录指针定位在pos位置。

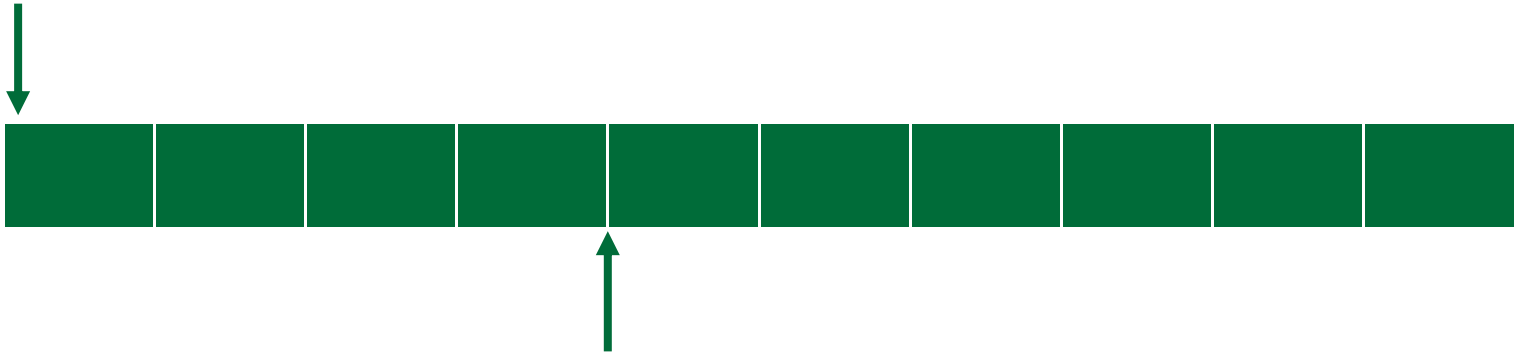
  - 通过它们可以获取和操作文件记录指针，实现RandomAccessFile与众不同的定位随机访问。

4．RandomAccessFile的常用访问方式

- 创建RandomAccessFile对象时，需要指定文件的访问方式，常用的包括下面两种：
- “r”：以只读方式打开指定文件。
- “rw”：以读、写方式打开指定文件，如果文件不存在则尝试创建该文件。

- RandomAccessFile没有只写的访问方式。

【例12-10】向data.dat文件中写入10个int型随机整数；从键盘上输入1~10这些序号，在控制台打印出对应数字。



12.6 Java输入流输出流汇总

- 节点流：直接与不同的物理存储打交道。
- 处理流：对节点流进行包装，或提供缓存、或提供数据的格式化读写、打印，总之使输入输出更简单便捷，执行效率更高。

12.6 Java输入流输出流汇总

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 节点流
  - 对内存进行读写的类（包括字符数组、字节数组、字符串）：CharArrayReader、CharArrayWriter、ByteArrayInputStream、ByteArrayOutputStream、StringReader、StringWriter。
  - 对文件进行读写的类：FileReader、FileWriter、FileInputStream、FileOutputStream。

类 型	字 符 流	字 节 流
File（文件）	FileReader FileWriter	FileInputStream FileOutputStream
Memory Array	CharArrayReader CharArrayWriter	ByteArrayInputStream ByteArrayOutputStream
Memory String	StringReader StringWriter	—
Pipe（管道）	PipedReader PipedWriter	PipedInputStream PipedOutputStream

http://blog.csdn.net/jingzi123456789

12.6 Java输入流输出流汇总

北京理工大学 BEIJING INSTITUTE OF TECHNOLOGY

- 处理流
  - 对对象进行读写的类：ObjectInputStream、ObjectOutputStream。
  - 按Java基本数据类型读写的类：DataInputStream、DataOutputStream。
  - 格式化打印输出的类： PrintWriter、PrintStream。
  - 读写时对数据进行缓存的类：BufferedReader、BufferedWriter、BufferedInputStream、BufferedOutputStream。
  - 按照一定的编码标准将字节流转换为字符流的类：InputStreamReader、OutputStreamWriter。

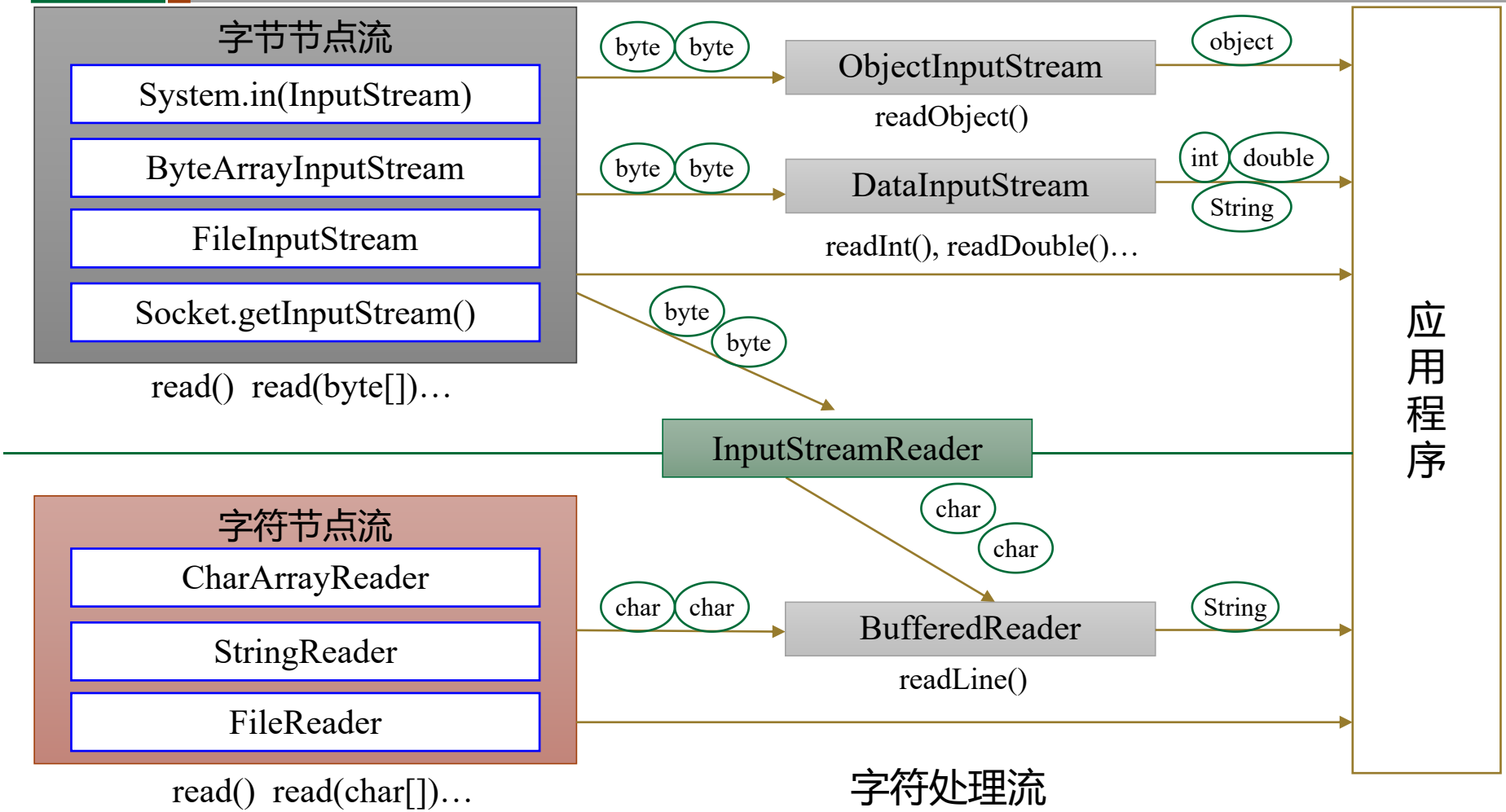
处理类型	字符流	字节流
Buffering	BufferedReader BufferedWriter	BufferedInputStream BufferedOutputStream
Filtering	FilterReader FilterWriter	FilterInputStream FilterOutputStream
Converting between bytes and character	InputStreamReader OutputStreamWriter	
Object Serialization	—	ObjectInputStream ObjectOutputStream
Data conversion	—	DataInputStream DataOutputStream
Counting	LineNumberReader	LineNumberInputStream
Peeking ahead	PusbackReader	PushbackInputStream
Printing	PrintWriter	PrintStream

http://blog.csdn.net/qq123456789

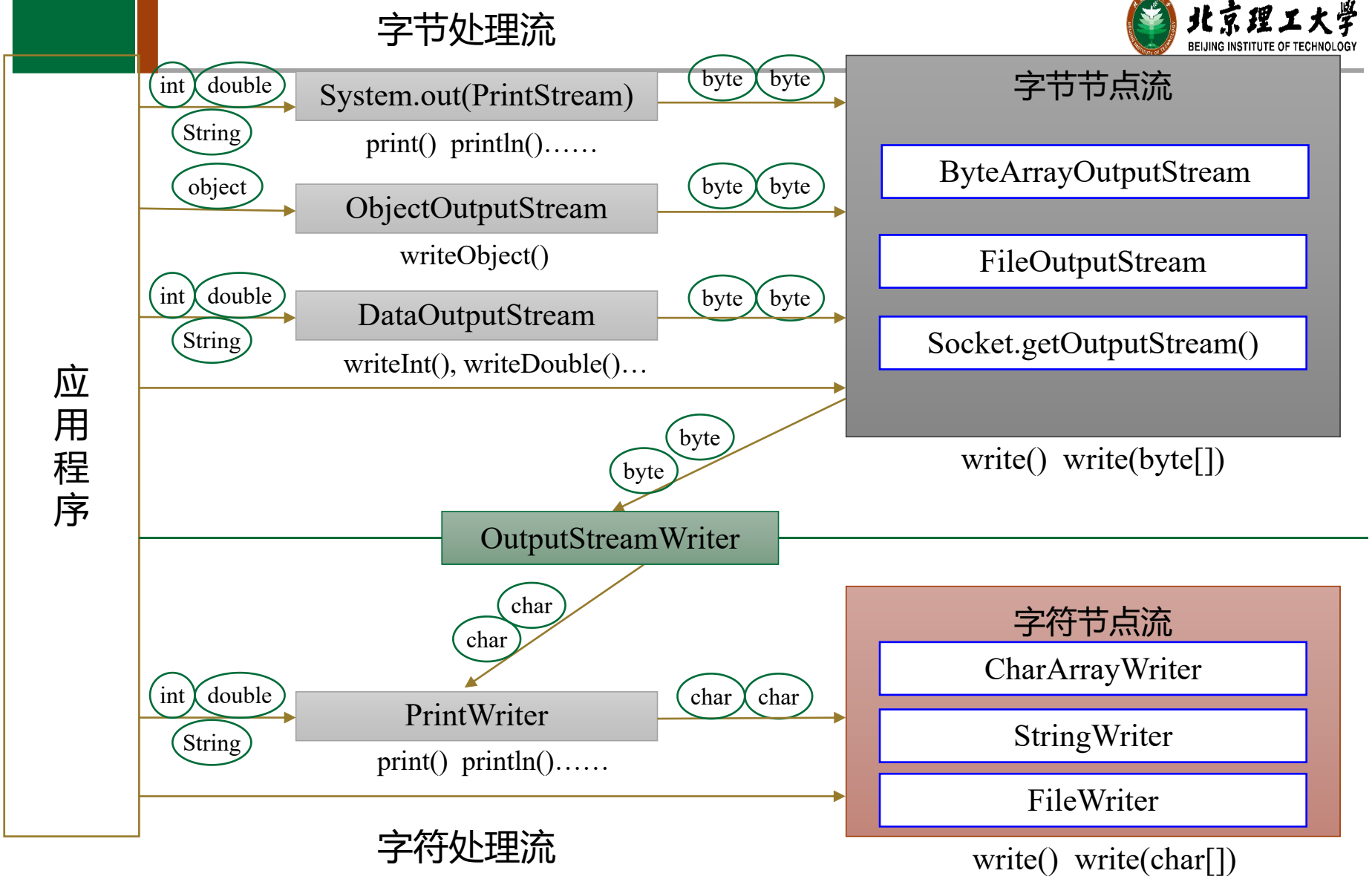
12.6 Java输入流输出流汇总

- 转换流：工作在字节流和字符流间，提供加入编码方式的字节数据与字符数据间的转换。

字节处理流


字节节点流
System.in(InputStream)
ByteArrayInputStream
FileInputStream
Socket.getInputStream()
read() read(byte[])…
byte
byte
ObjectInputStream
readObject()
object
byte
byte
DataInputStream
readInt(), readDouble()…
int
double
String
byte
byte
InputStreamReader
char
char
字符节点流
CharArrayReader
StringReader
FileReader
read() read(char[])…
char
char
BufferedReader
readLine()
String
应用程序

字符处理流


字节处理流
应用程序
int
double
String
System.out(PrintStream)
print() println()……
object
ObjectOutputStream
writeObject()
int
double
String
DataOutputStream
writeInt(), writeDouble()…
byte
byte
字节节点流
ByteArrayOutputStream
FileOutputStream
Socket.getOutputStream()
write() write(byte[])
byte
byte
OutputStreamWriter
char
char
int
double
String
PrintWriter
print() println()……
char
char
字符节点流
CharArrayWriter
StringWriter
FileWriter
write() write(char[])
字符处理流

System.in：InputStream类型的，代表标准输入流，默认状态对应于键盘输入。

- System.out：PrintStream类型的，代表标准输出流，默认状态对应于显示器输出。
- System.err：PrintStream类型的，代表标准错误信息输出流，默认状态对应于显示器输出。

System.out已经事先被包装成了PrintStream对象。
System.err同样也是PrintStream
System.in却是一个没有被包装过的未经加工的InputStream

Sample : Echo. java * **ChangeSystemOut**

重定向：

setIn(InputStream)

setOut(PrintStream)

setErr(PrintStream)

Sample：Redirecting.java

12.6 操作文件

- 除了存取文件的内容之外，文件管理还包括创建文件/目录、获取文件信息、复制移动删除文件等。
- Java SE 7在java.nio.file包中新增了Path接口、Paths类和Files类。

- 主要介绍File类

12.6.1 File类

- File类包含了对文件、目录进行操作的静态方法。
  - File对象代表一个文件或目录
    - File f=new File(“.”);将目录包装为file对象
    - File f=new File(“hello.txt”);将文件包装为file对象
- 主要作用:
  - 创建、删除 文件；
  - 重命名 文件
  - 判断文件的读写权限及是否 存在；
  - 设置和查询文件的最近修改时间 等;

list()　返回目录下的文件或者目录名。String[]

listFiles()　返回目录下的文件或者目录对象（File类实例）

list(FilenameFilter filter)返回指定当前目录中符合过滤条件的子文件或子目录。String[]

listFiles(FilenameFilter filter)返回指定当前目录中符合过滤条件的子文件或子目录。　（File类实例）

createNewFile()在指定位置创建一个空文件
mkdir()　在指定位置创建一个单级目录。
mkdirs()　在指定位置创建一个多级目录。
delete()　删除文件或者一个空文件夹
exists()　文件或文件夹是否存在
isFile()　是否是一个文件
isDirectory()　是否是一个目录，

示例：在C盘创建文件 Hello.txt Hello.txt Hello.txt， 如果存在则删除旧文件，不直接创建新的　**FileTest**